

Chapter 7

Abstraction, Interfaces, Packages, Access Modifiers, Encapsulation & Polymorphism

1. What Is Abstraction?

Abstraction is the process of hiding implementation details and showing only essential features of an object. It focuses on what an object does, not how it does it.

Real-Life Example: think of a car — you know how to drive it (start, stop, accelerate), but you don't know how the engine or braking system works internally. That's abstraction: hiding complex internal code and exposing only the necessary methods to the user.

Syntax of Abstraction

```
abstract class ClassName {  
    abstract void methodName(); // abstract method (no body)  
    void normalMethod() {      // concrete method  
        // code  
    }  
}
```

Example 1 — Abstract Class and Subclass

```
abstract class Animal {  
    abstract void sound();  
    void eat() {  
        System.out.println("Animals eat food");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound();  
        a.eat();  
    }  
}
```

Dog barks
Animals eat food

Explanation: `sound()` is abstract — it must be implemented in the subclass. `eat()` is a normal (concrete) method — it's inherited directly, no override needed.

Example 2 — Multiple Subclasses (Polymorphism with Abstraction)

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle();
        Shape s2 = new Rectangle();
        s1.draw();
        s2.draw();
    }
}
```

Drawing Circle

Drawing Rectangle

Explanation: the same abstract class `Shape` is used to define behavior for different shapes — a classic use of abstraction combined with polymorphism.

Example 3 — Abstract Class with Both Abstract and Concrete Methods

```
abstract class Vehicle {
    abstract void start();

    void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Bike extends Vehicle {
```

```

        void start() {
            System.out.println("Bike started");
        }
    }

    public class Main {
        public static void main(String[] args) {
            Vehicle v = new Bike();
            v.start();
            v.stop();
        }
    }

```

Bike started
Vehicle stopped

Rules for Abstract Classes

Rule	Description
Cannot create an object of an abstract class	Only subclass objects can be created.
Even one abstract method forces the class abstract	If a class has at least one abstract method, the class itself must be declared abstract.
Subclass must implement all abstract methods	Unless the subclass is also declared abstract.
Abstract classes CAN have constructors, variables, and normal methods	Abstraction doesn't mean 'empty class' — only specific methods are left unimplemented.

2. Interfaces

Example 4 — Basic Interface Implementation

```

interface Animal {
    void sound();
}

class Cat implements Animal {
    public void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Cat();
        a.sound();
    }
}

```

```
}  
}
```

Output: Cat meows. Explanation: the interface Animal defines WHAT should be done, and class Cat defines HOW it's done.

Example 5 — Multiple Interface Implementation

```
interface Printable {  
    void print();  
}  
interface Showable {  
    void show();  
}  
  
class Demo implements Printable, Showable {  
    public void print() {  
        System.out.println("Printing...");  
    }  
    public void show() {  
        System.out.println("Showing...");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.print();  
        d.show();  
    }  
}
```

Printing...

Showing...

Explanation: Java supports multiple inheritance using interfaces, not classes — a class can implement as many interfaces as needed.

Differences: Abstract Class vs Interface

Feature	Abstract Class	Interface
Keyword	abstract	interface
Methods	Can have abstract + non-abstract (concrete) methods	Only abstract (traditionally, till Java 7)
Variables	Can have instance variables	Only constants (public static final)
Multiple Inheritance	Not supported (a class extends only one class)	Supported (a class can implement many interfaces)
Constructor	Can have a constructor	Cannot have a constructor
Access	Uses the extends keyword	Uses the implements keyword

Advantages of Abstraction

Advantage	Description
Data Security	Hides implementation details.
Code Reusability	Same abstract class/interface reused by many subclasses.
Simplified Maintenance	Changing internal code doesn't affect outer code.
Improves Flexibility	Focuses on behavior, not structure.

Real-Time Example — Payment System

```
abstract class Payment {
    abstract void pay();
}

class CreditCardPayment extends Payment {
    void pay() {
        System.out.println("Paid using Credit Card");
    }
}

class UpiPayment extends Payment {
    void pay() {
        System.out.println("Paid using UPI");
    }
}

public class Main {
    public static void main(String[] args) {
        Payment p1 = new CreditCardPayment();
        Payment p2 = new UpiPayment();
        p1.pay();
        p2.pay();
    }
}
```

Paid using Credit Card

Paid using UPI

Explanation: both classes have different implementations for the same abstract method `pay()` — a real-world use of abstraction to model interchangeable payment methods.

Combined Program — Abstract Class + Interface (Shape/Color)

```
abstract class Shape {
    abstract void draw();
    void display() {
        System.out.println("Displaying Shape");
    }
}
```

```

interface Color {
    void fillColor();
}

class Circle extends Shape implements Color {
    void draw() {
        System.out.println("Drawing Circle");
    }
    public void fillColor() {
        System.out.println("Filling Red Color");
    }
}

public class Main {
    public static void main(String[] args) {
        Circle c = new Circle();
        c.draw();
        c.display();
        c.fillColor();
    }
}

```

Drawing Circle

Displaying Shape

Filling Red Color

Combined Program — Abstract Class + Interface (Student/Sports)

```

abstract class Student {
    abstract void study();
    void attendClass() {
        System.out.println("Student is attending class");
    }
}

interface Sports {
    void play();
}

class CollegeStudent extends Student implements Sports {
    void study() {
        System.out.println("Student is studying");
    }
    public void play() {
        System.out.println("Student is playing cricket");
    }
}

```

```

}

public class Main {
    public static void main(String[] args) {
        CollegeStudent s = new CollegeStudent();
        s.study();
        s.attendClass();
        s.play();
    }
}

```

Student is studying

Student is attending class

Student is playing cricket

Exercises — Abstraction & Interfaces

- Create a class Calculator and overload a method add() to handle int, float, and double values.
- Write a program with parent class Vehicle and child class Car, overriding move().
- Create a class Employee with overloaded display() methods that print different details.
- Write a runtime polymorphism program using Shape -> Circle, Rectangle, and Triangle.
- Demonstrate compile-time and runtime polymorphism in a single program.

3. Packages

A package in Java is a collection of related classes, interfaces, and sub-packages grouped together logically. Think of a package as a folder on your computer that stores similar types of files (classes).

```

package packageName;
public class ClassName {
    // ...
}

```

Example — Creating and Using a Package

```

// File: Student.java
package mypackage;

public class Student {
    public void display() {
        System.out.println("Hello from mypackage");
    }
}

// To use this class in another program:
import mypackage.Student;

```

```

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}

```

Advantages of Packages

Advantage	Description
Organization	Organizes related classes together.
Reusability	Classes in packages can be reused easily.
Name Conflict Avoidance	Two classes with the same name can exist in different packages.
Access Control	Provides protection using access modifiers.
Easier Maintenance	Code is easier to manage and debug.

Types of Packages in Java

Type	Description	Example
Built-in Packages	Provided by the Java API	java.util, java.io, java.lang
User-defined Packages	Created by the programmer	package studentapp;

Example — Using a Built-in Package

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.println("Hello " + name);
    }
}

```

Static Import

static import allows you to access static members (variables or methods) of a class directly, without using the class name as a prefix.

Syntax: `import static packageName.className.staticMember;`

```

import static java.lang.Math.*;

public class Main {
    public static void main(String[] args) {
        System.out.println(sqrt(25));
        System.out.println(pow(2, 3));
    }
}

```

5.0
8.0

Explanation: using static import, you can call static methods like `sqrt()` and `pow()` directly, without writing `Math.sqrt()` or `Math.pow()`.

4. Access Modifiers

Access Modifiers control the visibility (scope) of classes, methods, and variables.

Modifier	Scope	Accessible From
public	Everywhere	Within class, package, subclass, world
protected	Package + subclasses	Within the same package or a subclass
default (no keyword)	Package only	Within the same package
private	Class only	Within the same class only

Example — All Four Modifiers Together

```
class Person {
    private String name = "John";    // only within class
    public int age = 25;              // accessible everywhere
    protected String city = "Chennai"; // accessible in subclass
    String country = "India";         // package-level access
    (default)

    public void display() {
        System.out.println(name + ", " + age + ", " + city + ", " +
country);
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.display();
    }
}
```

Output: John, 25, Chennai, India

5. Encapsulation

Encapsulation is the process of binding data (variables) and methods together into a single unit (class), and restricting direct access to the data from outside. In simple words: encapsulation = data hiding + data protection.

Key Points

- Use private variables.

- Provide public getter and setter methods to access them safely.
- Prevents unauthorized access or modification.

Simple Encapsulation Example

```
class Student {
    private String name; // private data

    // getter method
    public String getName() {
        return name;
    }

    // setter method
    public void setName(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.setName("Ravi");
        System.out.println("Student Name: " + s.getName());
    }
}
```

Output: Student Name: Ravi. Explanation: data is hidden and accessed only through public methods.

Why Do We Need Getters and Setters?

In encapsulation, data members (variables) are made private, so they cannot be accessed directly from outside the class. To read or modify these private variables, we use public methods called getters (to read/get data) and setters (to write/set data).

Method	Purpose	Access
Getter	Used to get or return the value of a private variable	Usually public
Setter	Used to set or modify the value of a private variable	Usually public

Real-Time Encapsulation — BankAccount (Simple Version)

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
}
```

```

    }

    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        } else {
            System.out.println("Insufficient balance!");
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.deposit(1000);
        acc.withdraw(400);
        System.out.println("Remaining Balance: " + acc.getBalance());
    }
}

```

Output: Remaining Balance: 600.0. Explanation: the balance variable is hidden from direct access, ensuring data safety — it can only change through deposit() and withdraw(), which include their own validation.

Simple Example — Getter and Setter (Student, Two Fields)

```

class Student {
    private String name;
    private int age;

    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }

    public void setAge(int age) {
        this.age = age;
    }
    public int getAge() {
        return age;
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.setName("Asha");
        s1.setAge(20);

        System.out.println("Student Name: " + s1.getName());
        System.out.println("Student Age: " + s1.getAge());
    }
}

```

Student Name: Asha

Student Age: 20

The variables name and age are private (hidden). We access them using getter and setter methods.

Encapsulation Example — Bank Account with User Input (Account Number & Balance)

A fuller version of the BankAccount example: it stores both an account number and balance (both private), uses getters/setters for both, and reads all values live from the user via Scanner.

```

import java.util.Scanner;

class BankAccount {
    // Private data members
    private long accountNumber;
    private double balance;

    // Setter methods
    public void setAccountNumber(long accountNumber) {
        this.accountNumber = accountNumber;
    }
    public void setBalance(double balance) {
        this.balance = balance;
    }

    // Getter methods
    public long getAccountNumber() {
        return accountNumber;
    }
    public double getBalance() {
        return balance;
    }

    // Deposit method

```

```

    public void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount Deposited Successfully.");
    }

    // Withdraw method
    public void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Amount Withdrawn Successfully.");
        } else {
            System.out.println("Insufficient Balance!");
        }
    }
}

public class BankDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount account = new BankAccount();

        // User Input
        System.out.print("Enter Account Number: ");
        account.setAccountNumber(sc.nextLong());

        System.out.print("Enter Initial Balance: ");
        account.setBalance(sc.nextDouble());

        System.out.print("Enter Deposit Amount: ");
        double deposit = sc.nextDouble();
        account.deposit(deposit);

        System.out.print("Enter Withdraw Amount: ");
        double withdraw = sc.nextDouble();
        account.withdraw(withdraw);

        // Display Details
        System.out.println("\n----- Account Details -----");
        System.out.println("Account Number : " +
account.getAccountNumber());
        System.out.println("Available Balance : " +
account.getBalance());

        sc.close();
    }
}

```

Enter Account Number: 123456789
Enter Initial Balance: 10000
Enter Deposit Amount: 2000
Amount Deposited Successfully.
Enter Withdraw Amount: 5000
Amount Withdrawn Successfully.

----- Account Details -----
Account Number : 123456789
Available Balance : 7000.0

Inheritance in Encapsulation — the Private-vs-Protected Rule

Rule in Java (very clear): a private variable in a parent class CANNOT be accessed directly in a child class, even if the child class uses extends.

Option 1 (Best Practice) — Use Getter/Setter, Keep the Field private

```
class Parent {
    private int x = 10;

    public int getX() {
        return x;
    }
}

class Child extends Parent {
    void show() {
        System.out.println(getX()); // Works
    }
}
```

Option 2 — Use protected (If Direct Subclass Access Is Intended)

```
class Parent {
    protected int x = 10;
}

class Child extends Parent {
    void show() {
        System.out.println(x); // Works
    }
}
```

Both approaches work, but the private + getter/setter version is the best practice, since it keeps the field itself fully protected while still exposing a controlled way for subclasses (and anyone else) to read it.

Advantages of Encapsulation

Advantage	Description
Data Hiding	Sensitive data is protected using private fields.
Data Security	Prevents unauthorized modification.
Code Reusability	Methods provide controlled access.
Easy Maintenance	Changing one part doesn't affect others.
Improved Flexibility	You can control how data is set or read.

Exercises — Access Modifiers & Encapsulation

Access Modifier Exercises:

- Create a class Person with private, protected, and public variables and observe their accessibility in another class.
- Try accessing a private variable directly and through a method — note the difference.

Encapsulation Exercises:

- Create a class Employee with private fields: name, salary. Provide getter/setter methods.
- Write a program for BankAccount where balance can only be modified through deposit() and withdraw() methods.
- Create a real-world encapsulated class MobilePhone with private data like model, price.

6. Polymorphism in Java

The word Polymorphism means "many forms". In Java, polymorphism allows one object to behave in multiple ways depending on how it is used. In simple terms: the same method or operator behaves differently based on the object or data it acts on.

Real-Life Example: think of a person — as a teacher, they teach; as a parent, they care for children; as a citizen, they follow laws. The same person performs different roles in different contexts — that's polymorphism!

Types of Polymorphism in Java

Type	Description	Example
Compile-time Polymorphism	Method overloading — resolved at compile time	Multiple methods with the same name, different parameters
Runtime Polymorphism	Method overriding — resolved at runtime	Subclass provides its own version of a parent class method

Type 1: Compile-Time Polymorphism (Method Overloading)

When multiple methods have the same name but different parameters (type, number, or order) within the same class, it is called method overloading. The compiler determines which method to call based on the arguments passed.

Rules for Method Overloading

- Methods must have the same name.
- Must differ in: number of parameters, or type of parameters, or order of parameters.

- Return type alone doesn't count — return type doesn't matter for distinguishing overloaded methods.

Example — Method Overloading by Number of Parameters

```
class MathOperation {
    void add(int a, int b) {
        System.out.println("Sum: " + (a + b));
    }
    void add(int a, int b, int c) {
        System.out.println("Sum: " + (a + b + c));
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperation m = new MathOperation();
        m.add(5, 10);
        m.add(2, 3, 4);
    }
}
```

Sum: 15

Sum: 9

Type 2: Runtime Polymorphism (Method Overriding)

When a subclass provides its own version of a method already defined in the parent class, it is called method overriding. The method to be executed is determined at runtime, based on the actual object being referred to (not the reference type).

Rules for Method Overriding

- The method in the child and parent class must have the same name and same parameters.
- The method in the subclass must have equal or higher access (e.g., public/protected, not more restrictive).
- The overriding method cannot be less accessible, and cannot be static or final.

Example — Basic Method Overriding

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

```

}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}

```

Output: Dog barks. Explanation: at runtime, Dog's sound() method is executed — not Animal's — even though the reference type is Animal. This is Runtime Polymorphism.

Real-Life Example of Polymorphism — Shapes

```

class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Triangle extends Shape {
    void draw() {
        System.out.println("Drawing a Triangle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle();
        Shape s2 = new Triangle();
        s1.draw();
        s2.draw();
    }
}

```

Drawing a Circle

Drawing a Triangle

Explanation: the same method draw() behaves differently based on the actual object — Circle or Triangle — even though both are accessed through a Shape reference.

Exercises — Polymorphism

- Create a class Calculator and overload a method add() to handle int, float, and double values.
- Write a program with parent class Vehicle and child class Car, overriding move().
- Create a class Employee with overloaded display() methods that print different details.
- Write a runtime polymorphism program using Shape -> Circle, Rectangle, and Triangle.
- Demonstrate compile-time and runtime polymorphism in a single program.

7. Tips for This Chapter

1. Abstraction answers 'what can this object do', while encapsulation answers 'how is this object's data protected'. They're often taught together because both involve hiding something, but abstraction hides implementation complexity, while encapsulation specifically hides and guards data.
2. An abstract class is for closely related subclasses that should share some common code (like Vehicle.stop() being reused by every vehicle type). An interface is for unrelated classes that all need to guarantee the same capability (like Printable and Showable, which have nothing to do with each other otherwise).
3. If a class needs to inherit shared code AND guarantee multiple unrelated capabilities, that's exactly when you combine both — extends one abstract class, implements one or more interfaces, as shown in the Shape+Color and Student+Sports examples.
4. For encapsulation, the private + public getter/setter pattern is not just a style preference — it's what lets you add validation (like the withdraw() balance check) without ever letting outside code bypass it by touching the variable directly.
5. Remember the one hard inheritance-encapsulation rule: a private field is NEVER visible to a subclass, no matter what. If a subclass genuinely needs direct field access, the field must be protected (not private) — otherwise, the parent must expose a getter.
6. For polymorphism, the fastest way to tell compile-time vs runtime apart: if Java can figure out which method to call just by reading the method call and its arguments (overloading), it's compile-time. If Java has to check what actual object is behind the reference at the moment the program runs (overriding), it's runtime.
7. A very common exam trap: Animal a = new Dog(); a.sound(); — the TYPE of the reference (Animal) does not decide which method runs; the ACTUAL object (Dog) does. Always trace runtime polymorphism by asking 'what's really being pointed to', not 'what's the reference declared as'.
8. When overloading, only the parameter list matters (count, type, or order) — changing just the return type is not enough to make two methods 'different' to Java, and will actually cause a compile error if everything else matches.
9. Packages and access modifiers work together: default (no modifier) access only makes sense once you understand packages, since 'package-only' access is meaningless without knowing what a package boundary actually is. Think of a package as the 'next circle out' from a class — private is the tightest circle, then default (package), then protected (package + subclasses), then public (everywhere).